API System Documentation: Account and Message Services

Jaehoon Song

May 18, 2026

1 Introduction

This document provides the formal specifications for the Account and Message management system, acting as the backend for a hypothetical social media application. It details the HTTP endpoints, expected request formats, business validation logic, database integration patterns, and standardized response structures required for the implementation of the backend services.

1.1 Architecture

The system is built utilizing a robust **3-Layer Architecture** to enforce separation of concerns:

- **Controller Layer:** Manages all incoming HTTP requests via the Javalin framework, parses JSON payloads using Jackson ObjectMapper, routes requests to appropriate services, and returns formatted JSON responses with corresponding HTTP status codes.
- **Service Layer:** Encapsulates all business logic and strict validation rules. It ensures data integrity before engaging the database, providing a barrier between raw external input and internal state.
- **Data Access Object (DAO) Layer:** Abstracts all direct database interactions. Utilizing JDBC and parameterized `PreparedStatement`s, it executes SQL queries safely against an in-memory H2 database, returning entity objects back to the services.

1.2 Database Entities

Figure 1 shows a compact **Crow's Foot** (IE) style ERD: their relationships are defined as "each **Message** has exactly one **Account** (`posted_by`)" and "each **Account** has zero or more **Message**s".

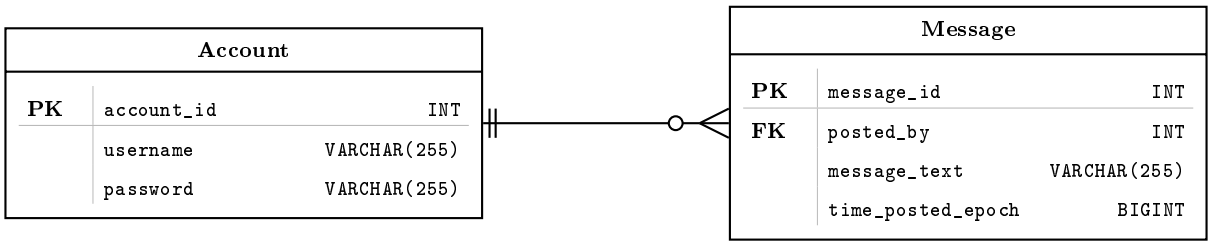

Figure 1: Crow's Foot ERD

Physical mapping (H2). `account_id` and `message_id` are `INT` primary keys with `AUTO_INCREMENT`; `posted_by` is an `INT` foreign key to `account(account_id)`; `username` is `VARCHAR(255) UNIQUE`.

2 Account Management

2.1 User Registration

A new account is created through a `POST` request to `localhost:8080/register`.

- **Request Body:** A JSON representation of an Account is provided, excluding the `account_id`.
- **Success Criteria:** Registration is completed if the username is not blank, the password contains at least four characters, and the username does not already exist in the database.
- **Successful Response:** A status of `200 OK` is returned. The response body contains the JSON representation of the Account, including the auto-generated `account_id`.
- **Error Handling:** If any validation failure occurs (including a duplicate username detection), a `400 Bad Request` (Client Error) status is returned. *(Note: Consolidated to 400 to match standard integration test expectations).*

2.2 User Login

Authentication is verified via a `POST` request to `localhost:8080/login`.

- **Request Body:** A JSON representation of an Account is provided, excluding the `account_id`.
- **Success Criteria:** Login is successful if the provided username and password match an exact, existing record in the database.
- **Successful Response:** A status of `200 OK` is returned. The response body contains the JSON of the authenticated account, including the `account_id`.
- **Error Handling:** If the credentials do not match any existing account, a `401 Unauthorized` status is returned.

3 Message Management

3.1 Create New Message

A new post is submitted via a `POST` request to `localhost:8080/messages`.

- **Request Body:** A JSON representation of a message is provided, excluding the `message_id`.
- **Success Criteria:** The message is persisted if the `message_text` is not blank, is 255 characters or less, and the `posted_by` field refers to a valid, existing user account.
- **Successful Response:** A status of `200 OK` is returned. The response body contains the complete JSON of the message, including the newly generated `message_id`.
- **Error Handling:** If validation fails for any reason (e.g., text too long, user doesn't exist), a `400 Bad Request` (Client Error) status is returned.

3.2 Retrieve All Messages

All messages are retrieved via a `GET` request to `localhost:8080/messages`.

- **Functionality:** Retrieves all global posts currently stored in the system.
- **Successful Response:** A status of `200 OK` is returned. The response body contains a JSON list of all messages retrieved from the database. An empty list (`[]`) is returned if no messages exist.

3.3 Retrieve Message by ID

A specific message is retrieved via a `GET` request to `localhost:8080/messages/{message_id}`.

- **Functionality:** Retrieves a single post matching the exact ID path parameter.
- **Successful Response:** A status of `200 OK` is returned. The response body contains the JSON representation of the identified message. If the message is not found, the response body remains completely empty.

3.4 Delete Message by ID

A message is removed via a `DELETE` request to `localhost:8080/messages/{message_id}`.

- **Functionality:** The identified message is permanently removed from the database.
- **Successful Response:** A status of `200 OK` is returned. If the message existed prior to deletion, the body contains the JSON representation of the **now-deleted message**. If the message did not exist, the body remains empty to maintain idempotency across multiple calls.

3.5 Update Message by ID

A message text is updated via a `PATCH` request to `localhost:8080/messages/{message_id}`.

- **Request Body:** A JSON payload containing a single field with the new `message_text` value. No other fields are guaranteed to be provided.
- **Success Criteria:** The update is executed if the `message_id` already exists in the database, the new text is not blank, and the new text length is 255 characters or fewer.
- **Successful Response:** A status of `200 OK` is returned, and the body contains the **fully updated message object** (including `message_id`, `posted_by`, `message_text`, and `time_posted_epoch`).
- **Error Handling:** If validation or existence checks fail, a `400 Bad Request` (Client Error) status is returned.

3.6 Retrieve Messages by Account ID

All messages posted by a specific user are retrieved via a `GET` request to `localhost:8080/accounts/{account_id}/messages`.

- **Functionality:** Filters global messages, returning only those authored by the provided user.
- **Successful Response:** A status of `200 OK` is returned. The response body contains a JSON list of all messages associated with the specified `account_id`. An empty list (`[]`) is returned if no such messages are found or if the account itself does not exist.